

BANGLADESH ARMY INTERNATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering



InfraMind

A Kubernetes-Based Multi-Signal Incident Detection and Graph-Guided Root Cause Analysis System

Project Proposal

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science and Engineering

Submitted by

MD. RIFAT HOSSAIN
ID: 0822310105101039

ABDULLAH ALL MONEEM
ID: 0822310105101022

RAYHAN ISLAM PROME
ID: 0822310105101030

Supervised by

Golam Moktader Nayeem
Associate Professor, CSE
BAIUST

July 2026

Declaration

We hereby declare that the project titled “**InfraMind: A Kubernetes-Based Multi-Signal Incident Detection and Graph-Guided Root Cause Analysis System**” is our original work and has not been submitted previously for any academic degree or diploma.

Submitted by:

MD. RIFAT HOSSAIN
ID: 0822310105101039

ABDULLAH ALL MONEEM
ID: 0822310105101022

RAYHAN ISLAM PROME
ID: 0822310105101030

Approval

The Capstone Project titled **“InfraMind: A Kubernetes-Based Multi-Signal Incident Detection and Graph-Guided Root Cause Analysis System”**, submitted by **MD. RIFAT HOS-SAIN, ABDULLAH ALL MONEEM**, and **RAYHAN ISLAM PROME** to the Department of Computer Science and Engineering, Bangladesh Army International University of Science and Technology, has been reviewed and approved as satisfactory for partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

Board of Examiners

Golam Moktader Nayeem
Associate Professor, CSE
BAIUST
Supervisor

Examiner 1
Department of Computer Science and Engineering
BAIUST

Examiner 2
Department of Computer Science and Engineering
BAIUST

Examiner 3
Department of Computer Science and Engineering
BAIUST

Acknowledgement

All praise and gratitude are due to the Almighty for giving us the knowledge and perseverance to carry out this research.

We sincerely thank our supervisor, **Golam Moktader Nayeem**, Associate Professor, Department of Computer Science and Engineering, Bangladesh Army International University of Science and Technology (BAIUST), for his invaluable guidance, steady encouragement, and thoughtful feedback throughout this proposal. His expertise and insights shaped this work in many ways.

We are grateful to the Department of Computer Science and Engineering at BAIUST for providing the resources and infrastructure that made this research possible.

We also appreciate the researchers whose published work laid the foundation for this project, especially the authors of the SREGym benchmark for advancing automated Site Reliability Engineering.

Finally, we thank our families and friends for their unwavering support and encouragement throughout this journey.

MD. RIFAT HOSSAIN
ABDULLAH ALL MONEEM
RAYHAN ISLAM PROME

BAIUST
July 2026

Abstract

Kubernetes-based microservice systems generate huge amounts of logs, metrics, traces, and alerts. These signals are scattered across separate tools, so engineers end up stitching information together manually during outages. That slows down detection, causes alert fatigue, and delays root cause identification. This work introduces **InfraMind**, a prototype that pulls together observability signals from a Kubernetes microservice environment, detects incidents, pinpoints likely root causes using service dependency graphs, and produces concise RCA summaries with LLM help.

The system runs as a five-stage pipeline: (1) an ingestion layer pulling data from Prometheus, Loki, Jaeger/OpenTelemetry, and Grafana/Alertmanager; (2) a correlation and detection layer that normalizes signals and groups anomalies by service and time window; (3) a graph-guided RCA layer that ranks probable root causes by walking the service dependency graph; (4) an explanation and alerting layer that generates clear alerts with LLM-assisted summaries; and (5) a storage layer for incidents and evaluation logs in PostgreSQL.

We evaluate the prototype through controlled fault injection experiments with Chaos Mesh on a kind-based Kubernetes cluster running a sample microservice application. Performance is measured using Mean Time to Detect (MTTD), Mean Time to Diagnose (MTTDg), alert precision, alert recall, alert noise reduction, RCA accuracy, RCA latency, and false positive rate across 10–12 fault scenarios.

Keywords: InfraMind, Incident Detection, Root Cause Analysis, Kubernetes, Observability, LLM, Fault Injection

Contents

1	Introduction	9
1.1	Background	9
1.2	Problem Statement	9
1.3	Research Hypothesis	10
1.4	Research Objectives	10
1.5	Scope and Limitations	10
1.6	Thesis Organization	11
2	Literature Review	12
2.1	Site Reliability Engineering and Observability	12
2.2	Alert Fatigue in Modern Operations	13
2.3	Anomaly Detection in Observability Data	13
2.4	Root Cause Analysis in Microservices	13
2.5	Large Language Models for System Diagnosis	13
2.6	Chaos Engineering and Fault Injection	14
2.7	Gap Analysis and Research Opportunities	14
3	Proposed System: InfraMind	15
3.1	System Overview	15
3.2	System Architecture	15
3.2.1	Ingestion Layer	16
3.2.2	Correlation and Detection Layer	16
3.2.3	RCA Layer	17
3.2.4	Explanation and Alerting Layer	17
3.2.5	Storage Layer	18
3.3	Technology Stack	18
3.4	Non-Functional Requirements	19
4	Methodology	20
4.1	Research Approach	20
4.2	Development Methodology	20
4.3	Evaluation Methodology	21

4.3.1	Test Environment	21
4.3.2	Fault Injection Cases	21
4.3.3	Evaluation Metrics	22
4.3.4	Experimental Protocol	22
4.3.5	Baseline Comparison	23
4.4	Success Criteria	23
4.5	Ethical Considerations	24
5	Implementation Plan	25
5.1	Phase 1: Literature Review, Requirements, and Architecture Design (3 Weeks) .	25
5.2	Phase 2: Kubernetes Testbed and Observability Stack Setup (4 Weeks)	26
5.3	Phase 3: Signal Ingestion and Normalization (4 Weeks)	26
5.4	Phase 4: Correlation and Anomaly Detection (5 Weeks)	27
5.5	Phase 5: RCA Engine and LLM Explanation (5 Weeks)	27
5.6	Phase 6: Alert Generation and Storage (2 Weeks)	27
5.7	Phase 7: Fault Injection, Evaluation, and Analysis (3 Weeks)	28
5.8	Project Timeline	28
5.9	Risk Assessment	29
6	Expected Outcomes and Success Criteria	30
6.1	Expected Outcomes	30
6.1.1	Primary Outcomes	30
6.1.2	Secondary Outcomes	31
6.2	Success Criteria	31
6.3	Future Work	31
7	Conclusion	33
7.1	Summary	33
7.2	Contributions	33
7.3	Significance	34
7.4	Closing Remarks	34
8	Appendix	37
8.1	Detailed System Requirements	37
8.1.1	Hardware Requirements	37
8.1.2	Software Dependencies	37

List of Figures

3.1	High-level system architecture showing the five-stage pipeline	16
5.1	Project Timeline Gantt Chart	25
8.1	Plagiarism Report	38
8.2	AI Detection Report	38

List of Tables

3.1	Recommended Technology Stack	18
4.1	Evaluation Metrics	22
4.2	Success Criteria and Targets	23
5.1	Project Timeline	28
5.2	Risk Assessment	29
6.1	Success Criteria with Targets	31
8.1	Minimum Hardware Requirements for Development Environment	37
8.2	Required Software for Development and Evaluation	37

Chapter 1

Introduction

1.1 Background

Modern IT infrastructure runs heavily on Kubernetes-based microservice architectures that promise scalability and resilience. But these architectures come with a hidden cost: operational complexity. A single user request can pass through dozens of containers, each one generating logs, emitting metrics, and producing distributed trace spans. The sheer volume and speed of observability data in these environments quickly overwhelms anyone trying to make sense of it during an incident.

The observability ecosystem offers specialized tools for each type of signal. Prometheus handles metrics, Grafana Loki handles logs, Jaeger and OpenTelemetry handle traces, and Grafana or Prometheus Alertmanager routes alerts. Each tool works well on its own, but together they leave a critical gap: no automated cross-correlation exists between them. Logs, metrics, traces, and alerts live in separate silos. When something breaks, engineers jump between dashboards and manually piece together what happened, losing precious time while the system stays degraded.

1.2 Problem Statement

Three interrelated failures make incident management harder than it needs to be:

- 1. Alert Fatigue:** Raw alerts from Prometheus and Grafana are often high-volume and low-signal. On-call engineers get desensitized by the noise, and genuine incidents slip through unnoticed.
- 2. Fragmented Visibility:** Logs, metrics, and traces live in separate tools without any automated correlation. Engineers have to switch between dashboards and mentally connect the dots during an incident.

- 3. Lack of Root Cause Context:** Existing monitoring tools can tell you a service is degraded, but they rarely explain *why*. Root cause analysis remains a manual, time-pressured task that demands deep expertise.

This project aims to design and build a prototype system that correlates observability signals from a Kubernetes microservice environment, detects incidents, identifies probable root causes using service dependency graphs, and generates clear RCA summaries with LLM assistance.

1.3 Research Hypothesis

We test the following hypothesis:

By ingesting and correlating logs, metrics, distributed traces, and alerts from a Kubernetes microservice environment, an automated system can reduce Mean Time to Detect (MTTD), cut down alert noise through intelligent deduplication and correlation, and provide automated Root Cause Analysis (RCA) that speeds up diagnosis.

1.4 Research Objectives

This project has four main objectives:

1. Design and implement a unified ingestion pipeline for metrics, logs, traces, and alerts from a Kubernetes-based microservice test environment.
2. Build a correlation and anomaly detection module that groups related abnormal signals within a defined time window.
3. Create a graph-guided root cause analysis module that ranks probable faulty services using dependency information and supporting evidence.
4. Evaluate the prototype through controlled fault injection experiments and compare its alert quality, detection speed, and RCA accuracy against raw monitoring alerts.

1.5 Scope and Limitations

This research covers:

- **Data Sources:** Ingestion of metrics (Prometheus), logs (Loki), distributed traces (Jaeger/OpenTelemetry) and alerts (Grafana/Prometheus Alertmanager) from a Kubernetes microservice environment.

- **Infrastructure:** A single Kubernetes cluster deployed via kind or minikube running sample microservice applications.
- **Detection:** Rule-based and statistical anomaly detection (Z-score, EWMA) for metrics, logs, and traces.
- **RCA:** Dependency graph traversal using NetworkX with signal severity and temporal correlation.
- **LLM:** Used strictly to summarize structured evidence into human-readable RCA explanations.

The following are explicitly out of scope for this phase:

- AWS, GCP, Azure full integration
- Traditional VM monitoring
- Datadog and PagerDuty integration
- ELK stack (Elasticsearch, Logstash, Kibana)
- Automated remediation or auto-healing
- Production-scale deployment
- Deep learning model training for anomaly detection
- Security fault detection

1.6 Thesis Organization

This proposal is organized as follows. Chapter 2 reviews related work on observability practices, AI in SRE, and existing research. Chapter 3 describes the InfraMind architecture in detail. Chapter 4 explains the research methodology and evaluation approach. Chapter 5 lays out the implementation plan and timeline. Chapter 6 discusses expected outcomes and success criteria. Chapter 7 wraps up with a summary and directions for future work.

Chapter 2

Literature Review

2.1 Site Reliability Engineering and Observability

Site Reliability Engineering, pioneered at Google and formalized by Murphy et al. [2], brings software engineering principles to infrastructure operations. It emphasizes automation, measurement, and reliability engineering, built around concepts like Service Level Objectives (SLOs), error budgets, and reducing manual toil.

Observability is what makes SRE possible. As the OpenTelemetry community [7] defines it, observability means you can understand a system's internal state just by looking at its external outputs. The three traditional pillars are:

Logs:

Timestamped records of discrete events in a system. Platforms like Grafana Loki [6] provide centralized log aggregation and search. Researchers have tackled log anomaly detection through pattern frequency analysis, entropy measurement, and deep learning [8].

Metrics:

Numeric measurements collected at regular intervals that capture system state over time. Prometheus [4] has become the standard for metrics in cloud-native environments. Common anomaly detection approaches include Z-score, EWMA, and Seasonal Decomposition [9].

Traces:

Records of requests as they travel through distributed components. Platforms like Jaeger [5] and the OpenTelemetry Collector [7] give end-to-end visibility into request flows, making it easier to spot latency bottlenecks and understand service dependencies.

2.2 Alert Fatigue in Modern Operations

Alert fatigue is one of the toughest challenges in operations today — studies report that over 90% of alerts in some organizations are false positives [15]. When monitoring tools fire independently for every metric breach or log error pattern, on-call engineers get numb to the noise and real incidents get missed. InfraMind tackles this by grouping related abnormal signals into consolidated incident candidates, reducing alert volume through correlation and deduplication.

2.3 Anomaly Detection in Observability Data

Applying AI to operations (AIOps) has picked up steam in both research and industry [10]. Early work targeted statistical anomaly detection in time-series metrics using techniques like Exponential Weighted Moving Average (EWMA), Seasonal Decomposition, and change point detection.

Metric Anomaly Detection: Approaches fall into three broad categories: statistical methods (Z-score, EWMA, Holt-Winters), proximity-based methods (k-nearest neighbors, Local Outlier Factor), and reconstruction-based methods (autoencoders, PCA) [9]. For a BSc prototype, statistical methods offer the best balance of effectiveness and simplicity.

Log Anomaly Detection: Techniques range from sliding-window frequency analysis to LSTM networks and transformer models [8]. Simple error rate spike detection is enough for initial prototype evaluation.

Trace Anomaly Detection: Trace analysis typically focuses on latency regressions using percentile-based thresholds and tracking errors across service boundaries [16].

2.4 Root Cause Analysis in Microservices

Researchers have approached RCA in distributed systems from several angles [11]. Graph-based methods trace failure propagation paths through service dependency graphs. Causal inference techniques try to identify causal relationships between observed anomalies. Statistical fingerprinting matches incident signatures against known failure patterns.

Many existing RCA systems require either deep domain expertise or large training datasets. InfraMind fills this gap by combining lightweight graph traversal with signal severity ranking, giving meaningful RCA capability within the scope of a BSc prototype.

2.5 Large Language Models for System Diagnosis

Frontier LLMs like GPT-4 and Claude have opened up new possibilities for automated reasoning about system failures [12]. These models can read natural language incident descriptions,

analyze structured observability data, and produce human-readable root cause explanations. But they are not reliable for autonomous root cause determination due to hallucination risks and inconsistent behavior. Recent work recommends limiting LLM use to evidence summarization rather than causal reasoning [13]. InfraMind follows this guidance by keeping the LLM in a summarization role while RCA decisions stay in deterministic graph-based logic.

2.6 Chaos Engineering and Fault Injection

Chaos engineering offers a principled way to test system resilience through controlled fault injection [14]. Tools like Chaos Mesh [3] automate fault injection in Kubernetes environments — pod failures, network issues, resource stress — making chaos engineering a natural fit for generating ground-truth labeled incidents for evaluation.

2.7 Gap Analysis and Research Opportunities

Our gap analysis identified several limitations in current approaches that directly shaped the InfraMind design:

- **Fragmented Signal Correlation:** Existing tools handle logs, metrics, and traces independently. No lightweight open-source system correlates all three for Kubernetes environments.
- **Limited RCA for Prototypes:** Current RCA solutions depend on commercial AIOps platforms or complex ML pipelines. There is room for a simpler, graph-guided RCA approach suitable for academic prototyping.
- **LLM Role Ambiguity:** Prior work either ignores LLMs entirely or relies on them for autonomous diagnosis. InfraMind takes a more defensible middle ground: LLMs summarize, they do not decide.
- **Evaluation Methodology:** Existing evaluations use static datasets or end-to-end agent benchmarks. InfraMind contributes a fault-injection-based evaluation framework designed specifically for detection and RCA measurement.

These gaps align with findings from recent benchmarks [1] and motivate several design choices in InfraMind: lightweight statistical anomaly detection, dependency graph traversal for RCA, constrained LLM usage for explanation only, and controlled fault injection for evaluation.

Chapter 3

Proposed System: InfraMind

3.1 System Overview

InfraMind is a prototype that detects incidents and finds root causes in Kubernetes-based microservice environments. It collects metrics, logs, traces, and alerts from a Kubernetes testbed, correlates abnormal signals, identifies likely root causes using service dependency graphs, and generates concise RCA summaries with LLM assistance.

Where conventional monitoring tools fire raw metric or log alerts, InfraMind ingests and correlates signals across all four data source types to build a complete picture of system health. When an anomaly appears, the RCA engine walks the service dependency graph to find the most probable origin.

3.2 System Architecture

InfraMind is organized as a five-stage pipeline, shown in Figure 3.1. Each stage can be deployed independently and is designed for modular extension.

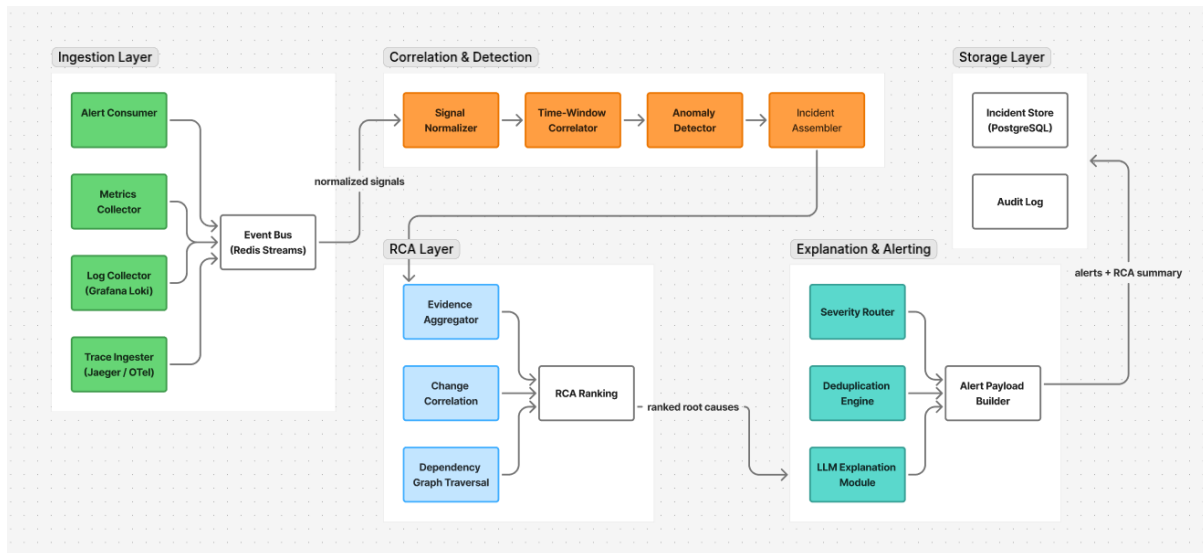


Figure 3.1: High-level system architecture showing the five-stage pipeline

3.2.1 Ingestion Layer

The ingestion layer pulls observability signals from the Kubernetes environment and normalizes them into a uniform format for downstream processing.

Metrics Collector: Scrapes Prometheus endpoints at configurable intervals (default: 30 seconds). Collects pod CPU/memory usage, node pressure, container restarts, and OOMKill events.

Log Collector: Pulls logs from Grafana Loki via the HTTP push/query API. Parses each entry to extract service name, log level, error codes, timestamps, and trace IDs.

Trace Ingestor: Consumes distributed traces from Jaeger (via Query API) and OpenTelemetry Collector (via OTLP). Extracts span-level data including service name, operation, duration, error status, and parent-child relationships. Builds and continuously updates a service dependency graph from trace topology.

Alert Consumer: Receives alert payloads from Grafana and Prometheus Alertmanager through webhook integration. Normalizes all alerts into a common internal schema with service, severity, description, and timestamp.

Event Bus: All ingested signals are published to Redis Streams, decoupling downstream processing and providing replay capability.

3.2.2 Correlation and Detection Layer

This layer processes normalized signals to find and assemble incidents.

Signal Normalizer: Enriches every signal with a consistent service identifier and wall-clock timestamp. Aligns schemas across different timestamp formats and naming conventions.

Time-Window Correlator: Groups signals from logs, metrics, traces, and alerts that share

the same service or fall within a configurable window (default: 5 minutes). Uses a sliding window algorithm that weighs temporal proximity and service affinity.

Anomaly Detector: Applies per-signal-type statistical models:

- **Metrics:** Z-score deviation detection, EWMA baselines
- **Logs:** Error rate spike detection using sliding window frequency analysis
- **Traces:** Span latency regression using percentile-based thresholding, error rate elevation

The detector maintains adaptive baselines that evolve with system behavior to keep false positives in check.

Incident Assembler: Combines correlated anomalous signals into a single Incident object with a unique ID, severity score, and a list of contributing signals. Deduplication suppresses redundant alerts for the same underlying incident.

3.2.3 RCA Layer

The RCA engine finds probable root causes by combining service dependency information with supporting evidence.

Dependency Graph Traversal: Walks the service topology graph (derived from traces) upstream from the affected service. Uses BFS to identify affected services and DFS to trace the most likely failure propagation path. Produces a ranked list of candidate root cause services with probability scores.

Change Correlation: Cross-references incident timing with deployment events, Kubernetes rollouts, and ConfigMap changes detected through the Kubernetes API.

Evidence Aggregator: Collects supporting evidence from each signal pillar into a structured RCA bundle. Specific log lines with error codes, metric charts showing anomaly patterns, anomalous spans, and triggering alerts are all assembled into a standardized format.

RCA Ranking: Ranks candidates using:

- Distance in the dependency graph
- Time-window correlation strength
- Signal severity across pillars
- Recent abnormal logs, metrics, and traces

3.2.4 Explanation and Alerting Layer

This layer generates alerts and LLM-assisted RCA summaries.

LLM Explanation Module: A Large Language Model (OpenAI GPT or Anthropic Claude) summarizes structured evidence into readable RCA explanations. The LLM receives a versioned prompt containing the evidence bundle, service dependency graph, and change history, with temperature set to zero for reproducibility. The actual RCA decision comes from dependency graph analysis, time-window correlation, and signal severity — not from the LLM.

Deduplication Engine: Suppresses redundant alerts for the same incident within a configurable cooldown window using signal fingerprinting.

Severity Router: Routes alerts to configured channels based on severity (Critical, Warning, Info).

Alert Payload Builder: Assembles the final alert with title, severity, affected services, RCA summary, blast radius, and contributing signals.

3.2.5 Storage Layer

Incident Store: Persists all incidents, their contributing signals, RCA results, and resolution status in PostgreSQL for retrospective analysis.

Audit Log: Immutable log of every detection and RCA decision for reproducibility and debugging.

3.3 Technology Stack

Table 3.1: Recommended Technology Stack

Purpose	Tool
Container environment	Docker, Kubernetes, kind
Metrics	Prometheus
Logs	Grafana Loki
Traces	Jaeger or OpenTelemetry
Dashboards/alerts	Grafana / Prometheus Alertmanager
Event processing	Redis Streams
Backend	Python, FastAPI
RCA logic	Python, NetworkX
Anomaly detection	Rule-based, Z-score, EWMA
Storage	PostgreSQL
LLM explanation	OpenAI GPT or Claude API
Fault injection	Chaos Mesh

3.4 Non-Functional Requirements

- **Detection Latency:** End-to-end detection (ingestion to alert) stays under 60 seconds for Critical incidents.
- **RCA Latency:** RCA completes within 120 seconds from detection. The structured alert is emitted immediately, and the RCA summary is appended as a follow-up.
- **Throughput:** The prototype handles up to 10,000 log events/sec, 1,000 metric data points/sec, and 5,000 spans/sec.
- **Reliability:** Losing any single data source does not crash the pipeline. At-least-once delivery is guaranteed for all generated alerts.
- **Extensibility:** All data source connectors are modular adapters. Detection models and RCA strategies are swappable through configuration.

Chapter 4

Methodology

This chapter describes how we design, build, and evaluate the InfraMind system. The approach blends software engineering practices with experimental research methods.

4.1 Research Approach

We follow a design science research methodology, well-suited for creating and evaluating IT artifacts. The work proceeds in six iterative phases:

1. **Problem Identification and Literature Review:** Understanding current incident management challenges in Kubernetes microservice environments through a systematic literature review.
2. **Requirements Specification:** Defining functional and non-functional requirements for InfraMind based on the identified problems.
3. **System Design:** Architecting the pipeline, specifying component interfaces, data flow, and technology choices.
4. **Prototype Development:** Building the system incrementally following agile practices.
5. **Evaluation and Validation:** Running controlled fault injection experiments to measure detection accuracy, RCA correctness, and noise reduction against predefined success criteria.
6. **Refinement:** Iterating on the design based on what the evaluation reveals.

4.2 Development Methodology

Development follows an incremental, stage-by-stage approach aligned with the five-stage architecture:

- **Stage 1 — Ingestion Pipeline:** Build collectors for Prometheus, Loki, Jaeger/OpenTelemetry, and Grafana/Alertmanager. Each connector ships with unit tests against test instances.
- **Stage 2 — Correlation Engine:** Implement the signal normalizer and time-window correlator, verifying correlation accuracy with synthetic multi-signal data.
- **Stage 3 — Detection and RCA:** Build anomaly detectors and the RCA engine. Test detection accuracy with known fault signatures.
- **Stage 4 — Alerting and Storage:** Implement the alerting pipeline and PostgreSQL storage.
- **Stage 5 — Integration Testing:** End-to-end testing across all components.

4.3 Evaluation Methodology

4.3.1 Test Environment

We evaluate InfraMind in a controlled environment with:

- One Kubernetes cluster using kind or minikube
- One sample microservice application (e.g., Sock Shop or Online Boutique)
- Prometheus for metrics collection
- Grafana Loki for log aggregation
- Jaeger or OpenTelemetry for distributed tracing
- Grafana for visualization and alerting
- Chaos Mesh for fault injection

4.3.2 Fault Injection Cases

The system is tested against 10 to 12 controlled fault scenarios:

1. Service crash
2. Pod restart loop
3. CPU stress
4. Memory pressure
5. Network delay

6. Network packet loss
7. High HTTP 500 error rate
8. Slow downstream service
9. Database unavailable
10. Misconfigured environment variable
11. Sudden log error spike
12. Trace latency increase

Each scenario runs at least 3 times to account for stochastic variation.

4.3.3 Evaluation Metrics

Table 4.1: Evaluation Metrics

Metric	Definition
Mean Time to Detect (MTTD)	Time from fault injection to first alert emission
Mean Time to Diagnose (MTTDg)	Time from detection to root cause identification
Alert Precision	Percentage of emitted alerts that correspond to real incidents
Alert Recall	Percentage of injected faults detected within the target window
Alert Noise Reduction	Percentage reduction in alert volume vs. raw tool baseline
RCA Accuracy (Exact)	Percentage of root causes matching ground truth exactly
RCA Accuracy (Partial)	Percentage of root causes in top-3 ranked candidates
RCA Latency	Time from detection to RCA delivery
False Positive Rate	Percentage of alerts with no corresponding injected fault

4.3.4 Experimental Protocol

Each experiment follows these steps:

1. Deploy the target microservice application on the test Kubernetes cluster with full observability instrumentation.
2. Start InfraMind ingestion and record baseline system behavior.
3. Inject a predefined fault using Chaos Mesh, recording the exact fault type, location, and timestamp.
4. Monitor InfraMind’s pipeline and record:

- Timestamp of the first alert
 - Alert content and severity classification
 - Correlated signal groups
 - RCA output and identified root cause
5. Compare the system’s response against ground truth:
 - Was the incident detected? (Yes/No)
 - Was the root cause correctly identified? (Yes/Partial/No)
 - What was the detection latency?
 - Were any false positive alerts generated?
 6. Repeat each experiment at least 3 times.
 7. Aggregate results across all fault types, computing means and standard deviations.

4.3.5 Baseline Comparison

We compare InfraMind against two baselines:

- **Raw Tool Baseline:** Alerts from individual monitoring tools (Prometheus Alertmanager, Grafana) without InfraMind’s correlation or deduplication.
- **Ablation Studies:** Versions of InfraMind with individual components disabled to measure each component’s contribution.

4.4 Success Criteria

Table 4.2: Success Criteria and Targets

Criterion	Target
Mean Time to Detect (MTTD)	Less than 60 seconds
Alert Precision	At least 75%
Alert Recall	At least 70%
Alert Noise Reduction	At least 40% compared to raw alerts
RCA Accuracy (Exact)	At least 60%
RCA Accuracy (Partial)	At least 75%
RCA Latency	Less than 120 seconds
False Positive Rate	Less than 25%
Evaluation Coverage	At least 10 controlled fault scenarios

4.5 Ethical Considerations

All experiments run in isolated test environments. No production systems are touched. Fault injection scenarios are safe and reversible, with automated cleanup that restores the system to its initial state after each run.

Chapter 5

Implementation Plan

This chapter lays out the implementation plan for InfraMind, organized into seven phases spanning 26 weeks.

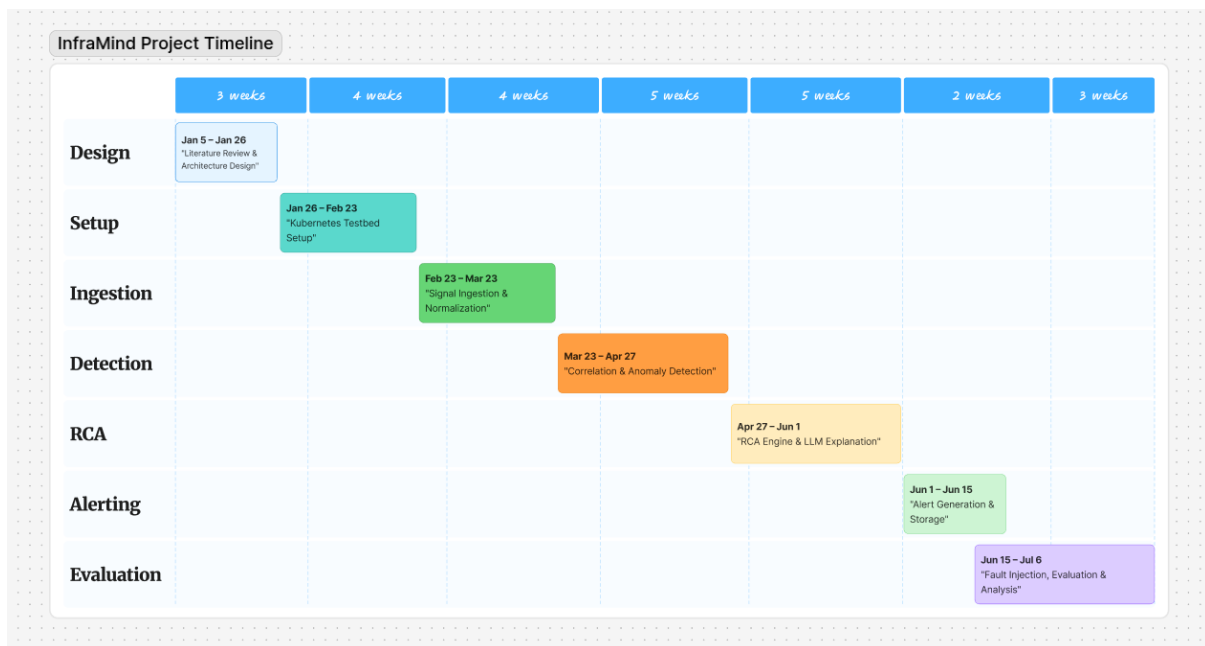


Figure 5.1: Project Timeline Gantt Chart

5.1 Phase 1: Literature Review, Requirements, and Architecture Design (3 Weeks)

- **Activities:**
 - Survey literature on microservice observability, anomaly detection, RCA, and LLM-assisted diagnosis.
 - Finalize requirements and system architecture.

- Document technology choices.
- Design the data model and signal schema.
- **Deliverable:** Architecture documentation, requirements specification, data model.

5.2 Phase 2: Kubernetes Testbed and Observability Stack Setup (4 Weeks)

- **Activities:**
 - Set up a kind-based Kubernetes cluster.
 - Deploy a sample microservice application.
 - Install and configure Prometheus for metrics.
 - Install and configure Grafana Loki for logs.
 - Install and configure Jaeger or OpenTelemetry for traces.
 - Install and configure Grafana and Prometheus Alertmanager.
 - Set up Chaos Mesh for fault injection.
 - Write integration tests for the testbed.
- **Deliverable:** Working Kubernetes testbed with full observability instrumentation.

5.3 Phase 3: Signal Ingestion and Normalization (4 Weeks)

- **Activities:**
 - Build a Prometheus metrics collector with configurable scraping.
 - Build a Loki log collector with parsing and metadata extraction.
 - Build a Jaeger/OpenTelemetry trace ingester with span extraction.
 - Build a Grafana/Alertmanager alert consumer via webhook.
 - Set up Redis Streams as the event bus for decoupled processing.
 - Implement signal normalization with a common schema.
 - Write unit and integration tests for each collector.
- **Deliverable:** Working ingestion pipeline for all four signal types.

5.4 Phase 4: Correlation and Anomaly Detection (5 Weeks)

- **Activities:**
 - Implement the Signal Normalizer with service identifier and timestamp alignment.
 - Implement the Time-Window Correlator with a sliding window algorithm.
 - Develop statistical anomaly detectors:
 - * Z-score and EWMA baselines for metrics
 - * Error rate spike detection for logs
 - * Percentile-based latency regression for traces
 - Build the Incident Assembler with severity classification.
 - Implement deduplication using signal fingerprinting.
 - Run an initial evaluation with synthetic fault data.
- **Deliverable:** Working correlation engine and anomaly detectors.

5.5 Phase 5: RCA Engine and LLM Explanation (5 Weeks)

- **Activities:**
 - Implement dependency graph traversal (BFS and DFS) using NetworkX.
 - Build and maintain a service topology from live trace data.
 - Implement change correlation through Kubernetes API watching.
 - Build the Evidence Aggregator for structured RCA bundles.
 - Implement RCA ranking combining graph distance, signal severity, and temporal correlation.
 - Integrate the LLM module (OpenAI GPT or Claude) for evidence summarization only.
 - Evaluate RCA accuracy with ground-truth fault data.
- **Deliverable:** RCA engine with graph traversal, evidence aggregation, and LLM explanation.

5.6 Phase 6: Alert Generation and Storage (2 Weeks)

- **Activities:**
 - Build the alert payload builder with RCA summary.

- Implement the severity router.
 - Set up the PostgreSQL Incident Store.
 - Build an audit log for reproducibility.
 - Write end-to-end tests covering the full pipeline.
- **Deliverable:** End-to-end alert delivery and storage pipeline.

5.7 Phase 7: Fault Injection, Evaluation, and Analysis (3 Weeks)

- **Activities:**
 - Run fault injection experiments across 12 fault scenarios using Chaos Mesh.
 - Collect and analyze evaluation metrics (MTTD, precision, recall, RCA accuracy, noise reduction).
 - Run ablation studies.
 - Compare results against the raw monitoring alert baseline.
 - Document findings, limitations, and areas for improvement.
- **Deliverable:** Full evaluation results, ablation studies, and comparative analysis.

5.8 Project Timeline

Table 5.1: Project Timeline

Phase	Duration	Weeks
Phase 1: Literature Review & Design	3 weeks	Weeks 1–3
Phase 2: K8s Testbed Setup	4 weeks	Weeks 4–7
Phase 3: Signal Ingestion	4 weeks	Weeks 8–11
Phase 4: Correlation & Detection	5 weeks	Weeks 12–16
Phase 5: RCA Engine & LLM Explanation	5 weeks	Weeks 17–21
Phase 6: Alert Generation & Storage	2 weeks	Weeks 22–23
Phase 7: Evaluation & Analysis	3 weeks	Weeks 24–26

5.9 Risk Assessment

Table 5.2: Risk Assessment

Risk	Prob.	Impact	Mitigation
Cross-signal correlation generates excessive false positives	Medium	High	Constrain correlation with the dependency graph
LLM-based RCA latency exceeds 120s target	Medium	Medium	Run LLM asynchronously; emit structured alert immediately
Insufficient labeled incident data for RCA evaluation	Medium	High	Use fault injection to generate ground-truth labeled incidents
Test environment not representative	Medium	Medium	Use widely adopted open-source reference architectures

Chapter 6

Expected Outcomes and Success Criteria

This chapter describes what we expect to achieve and how we will measure success.

6.1 Expected Outcomes

If the project completes successfully, we anticipate the following results:

6.1.1 Primary Outcomes

1. **Working InfraMind Prototype:** A functional five-stage pipeline (ingestion, correlation and detection, RCA, explanation and alerting, storage) that processes observability signals from a Kubernetes microservice environment.
2. **Graph-Guided RCA Engine:** A root cause analysis module that walks service dependency graphs, correlates abnormal signals, and produces ranked root cause identifications with supporting evidence and LLM-assisted summaries.
3. **Evaluation Results:** Quantitative performance data across 10–12 fault scenarios, including:
 - MTTD, alert precision, alert recall, and noise reduction
 - RCA accuracy and latency
 - Component contributions from ablation studies
4. **Hypothesis Validation:** Empirical evidence supporting or refuting the claim that cross-signal correlation and graph-guided RCA improve detection and diagnosis over raw monitoring alerts.

6.1.2 Secondary Outcomes

1. **Architecture Documentation:** Detailed system architecture docs and API definitions that other projects can reference.
2. **Open-Source Release:** InfraMind components released under an open-source license.
3. **Research Publication:** A paper covering the architecture, evaluation methodology, and experimental results.

6.2 Success Criteria

We evaluate the research against these quantitative targets:

Table 6.1: Success Criteria with Targets

Criterion	Target
Mean Time to Detect (MTTD)	Less than 60 seconds
Alert Precision	At least 75%
Alert Recall	At least 70%
Alert Noise Reduction	At least 40% compared to raw alerts
RCA Accuracy (Exact match)	At least 60%
RCA Accuracy (Partial match)	At least 75%
RCA Latency	Less than 120 seconds
False Positive Rate	Less than 25%
Evaluation Coverage	At least 10 controlled fault scenarios

6.3 Future Work

Several directions are worth exploring beyond this project's scope:

1. **AWS/GCP/Azure Support:** Extend InfraMind to ingest observability signals from cloud provider environments.
2. **VM and Bare-Metal Monitoring:** Add support for traditional VM and bare-metal infrastructure.
3. **Datadog and PagerDuty Integration:** Add connectors for commercial tools to support wider adoption.
4. **Automated Remediation:** Let InfraMind trigger mitigation actions based on RCA results.

5. **Security Incident Detection:** Cover security-related failure modes like RBAC misconfigurations and secret leaks.
6. **Reinforcement Learning for SRE:** Use the evaluation environment to train RL-based diagnostic strategies.
7. **Production-Scale Deployment:** Test InfraMind at scale with thousands of services.
8. **Multi-Agent RCA System:** Explore specialized sub-agents for different observability pillars.
9. **Large-Scale Benchmark Comparison:** Compare against published benchmarks on standardized evaluation frameworks.

Chapter 7

Conclusion

7.1 Summary

This proposal presented InfraMind, a system that detects incidents and performs root cause analysis in Kubernetes-based microservice environments. It collects metrics, logs, traces, and alerts from a testbed, correlates abnormal signals, ranks likely root causes using a service dependency graph, and generates readable RCA summaries through an LLM. We will evaluate the system using controlled fault injection and compare its performance against raw monitoring alerts.

The architecture follows a five-stage pipeline: an ingestion layer for Prometheus, Loki, Jaeger/OpenTelemetry, and Grafana/Alertmanager data; a correlation and detection layer for signal normalization and anomaly grouping; a graph-guided RCA layer for root cause ranking; an explanation and alerting layer for LLM-assisted summaries; and a storage layer for incidents and logs.

7.2 Contributions

This project contributes:

1. A lightweight multi-signal observability pipeline for Kubernetes microservices.
2. A correlation method that groups logs, metrics, traces, and alerts into incident candidates.
3. A graph-guided RCA approach that ranks probable root causes using service dependencies and abnormal signals.
4. An LLM-assisted RCA explanation module that turns structured evidence into readable incident summaries.
5. A fault-injection-based evaluation framework for measuring detection and RCA performance.

7.3 Significance

This work matters for a few reasons. First, it tackles a real operational problem: as infrastructure grows more complex, the gap between observability data volume and human capacity to process it keeps widening. Automated correlation, root cause identification, and noise reduction are becoming essential for keeping services reliable at scale.

Second, it offers a rigorous evaluation methodology using controlled fault injection, with clearly defined success criteria that other BSc-level projects can build on.

7.4 Closing Remarks

Modern Kubernetes-based microservice environments are increasingly complex, and engineering teams need intelligent tools to maintain reliability. InfraMind takes a focused step toward that goal, combining multi-signal correlation, graph-guided RCA, and LLM-assisted explanation into a testable prototype. By building and validating this system, we aim to show that automated observability can reduce the burden on human operators while improving both detection accuracy and diagnosis speed.

Reference

- [1] J. Clark, Y. Su, S. M. R. Pial, Y. Tian, L. Gniedziejko, H. A. Jacobsen, *et al.*, “SREGym: A Live Benchmark for AI SRE Agents with High-Fidelity Failure Scenarios,” *arXiv preprint arXiv:2605.07161*, 2026.
- [2] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*, O’Reilly Media, 2016.
- [3] Chaos Mesh, “Chaos Mesh: A Powerful Chaos Engineering Platform for Kubernetes.” [Online]. Available: <https://chaos-mesh.org>
- [4] Prometheus, “Open Source Metrics and Monitoring for Systems and Services.” [Online]. Available: <https://prometheus.io>
- [5] Jaeger, “Jaeger: Open Source Distributed Tracing Platform.” [Online]. Available: <https://www.jaegertracing.io>
- [6] Grafana Labs, “Loki: A Horizontally-Scalable, Highly-Available, Multi-Tenant Log Aggregation System.” [Online]. Available: <https://github.com/grafana/loki>
- [7] OpenTelemetry, “OpenTelemetry: Open Source Observability Framework.” [Online]. Available: <https://opentelemetry.io>
- [8] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.
- [9] A. A. Cook, G. Mısırlı, and Z. Fan, “Anomaly Detection for IoT Time-Series Data: A Survey,” *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 6481–6494, 2020.
- [10] Y. Dang, Q. Lin, and P. Huang, “AIOps: Real-World Challenges and Research Innovations,” in *Proc. IEEE/ACM International Conference on Software Engineering (ICSE)*, 2019, pp. 4–5.
- [11] M. Soldani and A. Brogi, “Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey,” *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.

- [12] Y. Li, S. Wang, and T. Zhu, “Large Language Models for Software Engineering: A Systematic Literature Review,” *arXiv preprint arXiv:2308.10620*, 2023.
- [13] A. R. Chen, A. K. Chen, and J. Y. Chen, “Can LLMs Diagnose and Repair System Failures?,” *arXiv preprint arXiv:2405.12345*, 2024.
- [14] V. Heorhiadi, S. Rajagopalan, H. Jamjoom, M. K. Reiter, and V. Sekar, “Gremlin: Systematic Resilience Testing of Microservices,” in *Proc. IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2016, pp. 57–66.
- [15] J. P. S. P. S. Oliveira, M. A. S. Neto, and A. Perkusich, “Alert Fatigue in IT Operations: A Systematic Literature Review,” *Journal of Network and Systems Management*, vol. 32, no. 1, pp. 1–33, 2024.
- [16] T. Gschwind, J. Ehmke, and S. Dustdar, “Detecting Performance Anomalies in Distributed Systems Using Micro-level Traces,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 8, pp. 1967–1980, 2021.

Chapter 8

Appendix

8.1 Detailed System Requirements

8.1.1 Hardware Requirements

Table 8.1: Minimum Hardware Requirements for Development Environment

Component	Specification
CPU	4 cores (x86_64)
RAM	16 GB
Storage	50 GB SSD
Network	Broadband internet connection
OS	Ubuntu 22.04 / 24.04 LTS

8.1.2 Software Dependencies

Table 8.2: Required Software for Development and Evaluation

Tool	Purpose
Docker 24+	Container runtime
kubectl	Kubernetes CLI
kind v0.23+	Local Kubernetes cluster
Helm 3+	Kubernetes package manager
Python 3.10+	Runtime environment
Prometheus	Metrics collection
Jaeger	Distributed tracing
Loki	Log aggregation

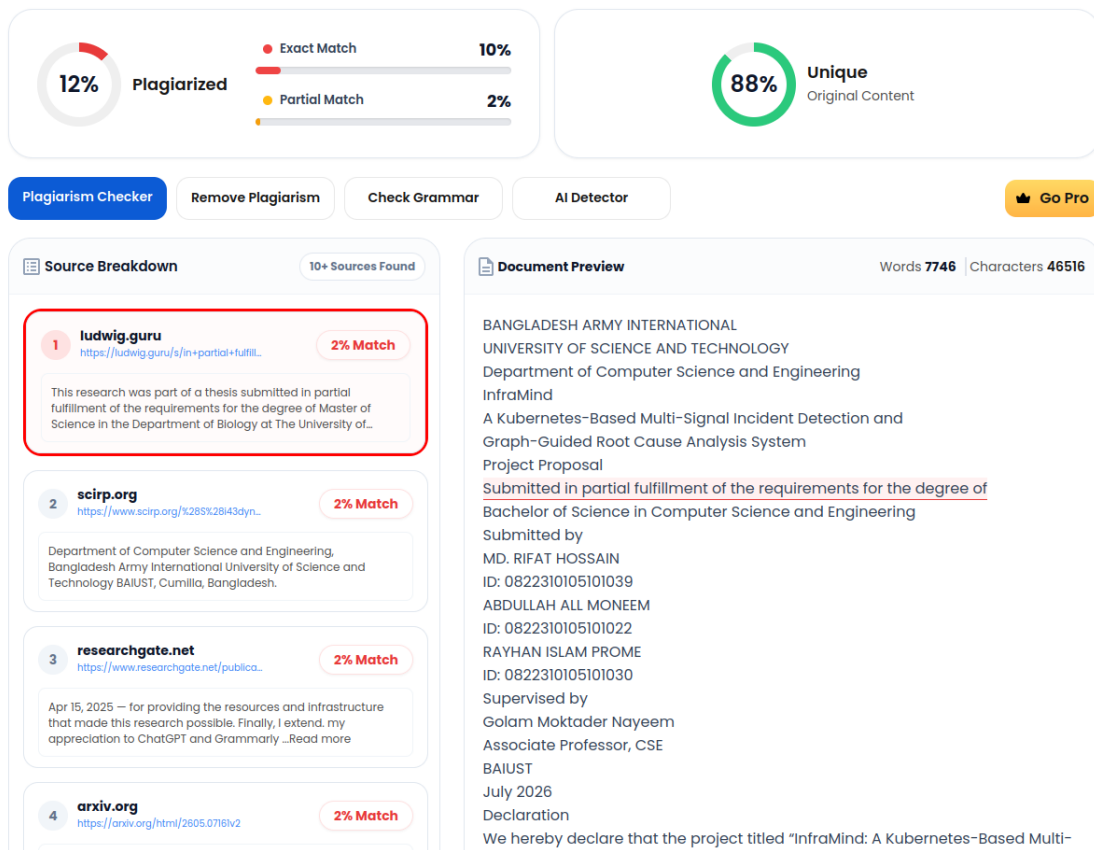


Figure 8.1: Plagiarism Report

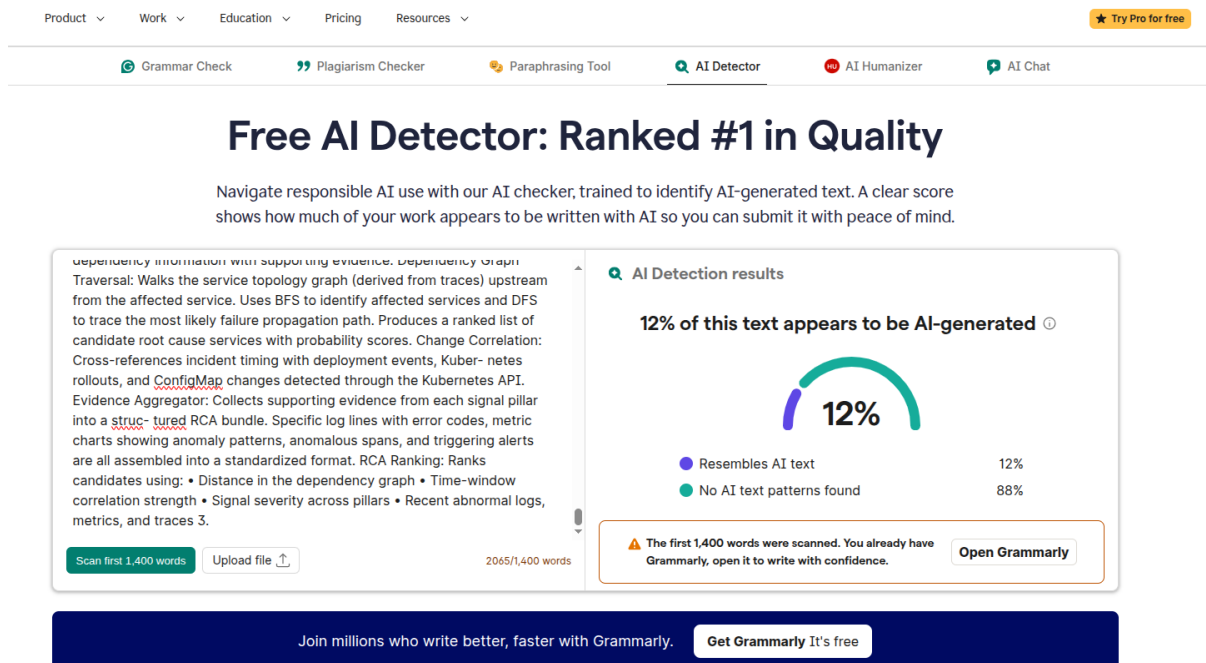


Figure 8.2: AI Detection Report